

Narzędzia i Aplikacje Jakarta EE

Uwierzytelnianie i Autoryzacja

Michał Wójcik

Kontrola dostępu

Uwierzytelnianie (authentication):

- weryfikacja tożsamości użytkownika,
- polega na sprawdzeniu informacji, którą zna/posiada tylko dany użytkownik:
  - np.: hasło, kod wysłany w wiadomości SMS, odcisk palca.

Autoryzacja (authorization):

- nadanie uprawnień do operacji i zasobów,
- najczęściej na podstawie ustaloną wcześniej tożsamość,
- wykorzystuje metadane przypisane użytkownikowi:
  - np.: przynależność do grup, przypisane role, nadane uprawnienia.

Model kontroli dostępu w Java EE

Kontrola dostępu w Java EE oparta jest na następującym **modelu**:

- RBAC (Role-Based Access Control) - kontrola dostępu oparta o role,
- dostęp do zasobu lub operacji wymaga posiadania określonej roli,
- każdemu użytkownikowi można przypisać wiele ról,
- jedną rolę można przypisać wielu użytkownikom,
- role odzwierciedlają rzeczywiste funkcje użytkownika w organizacji/firmie,
- nie przypisuje się uprawnień bezpośrednio do użytkowników,

Wyznaczanie ról:

- sposób wyznaczenia zbioru ról wybranego użytkownika jest dowolny,
- w najprostszym podejściu role są przypisywane bezpośrednio do użytkowników:
  - wystarczające w mniejszych projektach,
- w aplikacjach korporacyjnych dużej skali często występują hierarchiczne modele uprawnień:
  - np.: kohorty → grupy → role,
  - umożliwiają odzwierciedlenie struktury organizacyjnej.

Dostawcy tożsamości

Informacje o tożsamości użytkowników są zgromadzone poza aplikacją, np.:

- w bazie danych,
- na serwerze LDAP,
- w usłudze Active Directory,
- w pliku na dysku,
- u zewnętrznego dostawca tożsamości (np. Google, Facebook).

Serwery Java EE oferują moduły umożliwiające integrację z istniejącymi bazami użytkowników.

Uwierzytelnianie użytkownika

Uwierzytelnianie użytkownika:

- BASIC AUTH – ustawianie nazwy użytkownika i hasła zakodowanych w Base64, wysyłanych w każdym żądaniu za pomocą nagłówka `Authorization`,
- Formularz – dane uwierzytelniające użytkownika są zbierane za pomocą formularza i tworzone są odpowiednie obiekty sesji.
- Klucz API – użytkownik jest autoryzowany na podstawie klucza API dodawanego do każdego żądania (w nagłówku lub parametrze zapytania).

Są to nadal powszechnie stosowane, ale dość przestarzałe metody.

Utrzymywanie sesji

Jak zachować sesję w kolejnych żądaniach:

- Uwierzytelnianie oparte na plikach cookie – sesja uwierzytelnionego użytkownika jest identyfikowana za pomocą pliku cookie z identyfikatorem sesji wysyłanym z każdym żądaniem, plik cookie jest ustawiany w odpowiedzi (np. po uwierzytelnieniu za pomocą formularza).
- Uwierzytelnianie za pomocą parametrów zapytania – sesja uwierzytelnionego użytkownika jest identyfikowana za pomocą identyfikatora sesji przekazywanego z każdym żądaniem.
- Token – autoryzowany użytkownik jest identyfikowany za pomocą tokena utworzonego podczas procesu uwierzytelniania.

Te metody są nadal powszechnie używane. Na przykład JSF domyślnie używa teraz pliku cookie, ale wcześniej domyślnie używał parametru zapytania.

OAuth

OAuth (Open Authorization) — protokół autoryzacji umożliwiający aplikacjom innych firm dostęp do danych użytkownika bez ujawniania poświadczeń. Działa poprzez dostarczanie tokenów (access tokens) do aplikacji innych firm po wyrażeniu zgody przez użytkownika, umożliwiając ograniczony dostęp do określonych zasobów.

OAuth2 — zaktualizowana wersja OAuth z wieloma przepływami zaprojektowanymi dla różnych scenariuszy.

Role OAuth

Role w OAuth:

- **Właściciel zasobu** - (Resource Owner) Podmiot, który może udzielić dostępu do chronionego zasobu. Zazwyczaj jest to użytkownik końcowy.
- **Klient** - (Client) Aplikacja żądająca dostępu do chronionego zasobu w imieniu właściciela zasobu. Może to być aplikacja internetowa lub mobilna.
- **Serwer zasobów** - (Resource Server) Serwer hostujący chronione zasoby, do których dostęp jest możliwy tylko po uzyskaniu odpowiedniej autoryzacji.
- **Serwer autoryzacji** (Authorization Server) Serwer odpowiedzialny za uwierzytelnienie właściciela zasobu i zapewnienie tokenów dostępu klientowi po pomyślnym uwierzytelnieniu.

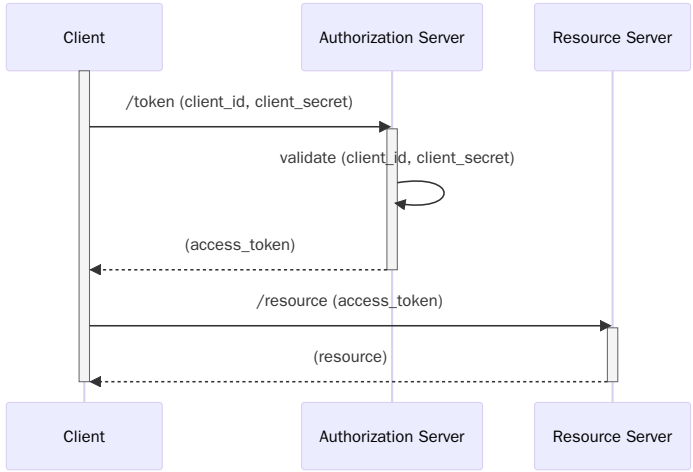
Przepływy OAuth2

Przepływy OAuth2:

- **Client Credentials Flow** — używany przez poufnych klientów (takich jak serwery) do uwierzytelniania i uzyskiwania tokenów dostępu na podstawie poświadczeń klienta (nie poświadczeń użytkownika).
- **Resource Owner Password Credentials Flow** — umożliwia klientowi bezpośrednie uzyskanie poświadczeń użytkownika (nazwa użytkownika/hasło) i wymianę ich na token dostępu. Mniej bezpieczny i należy go używać ostrożnie.
- **Authorization Code Flow** — przepływ używany przez aplikacje internetowe i mobilne, w których klient żąda autoryzacji od właściciela zasobu za pośrednictwem procesu przekierowania. Wymienia kod autoryzacji na token dostępu.
- **Implicit Flow** — używany głównie przez aplikacje internetowe jednostronicowe (SPA), w których tokeny dostępu są wydawane bezpośrednio, bez wymiany kodu autoryzacji. Jest mniej bezpieczny, ponieważ tokeny są ujawniane w przeglądarce.
- **Device Authorization Flow** — odpowiedni dla urządzeń o ograniczonych możliwościach wprowadzania danych (np. telewizory Smart TV), w których użytkownicy mogą uwierzytelniać się za pośrednictwem urządzenia pomocniczego, uzyskując token dostępu.

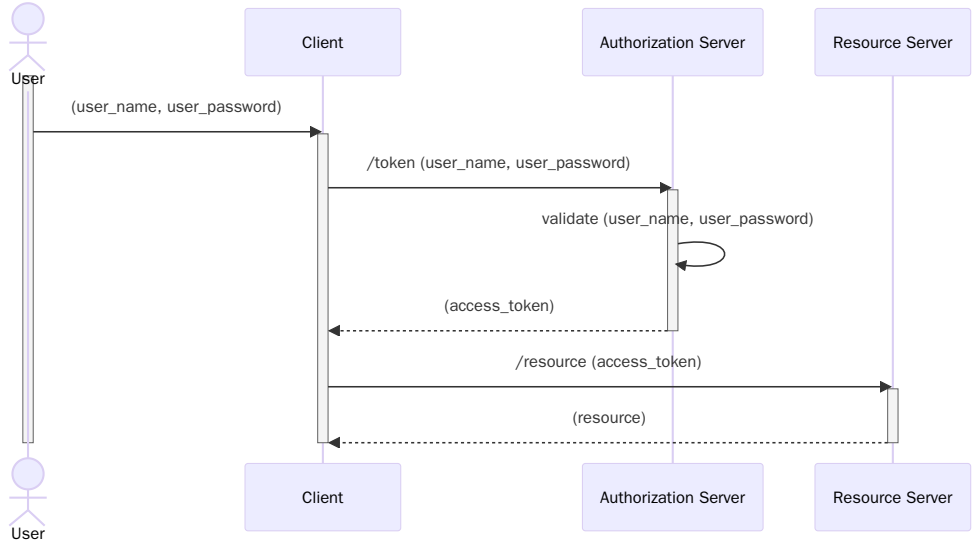
POLITECHNIKA GDAŃSKA

Client Credentials Flow



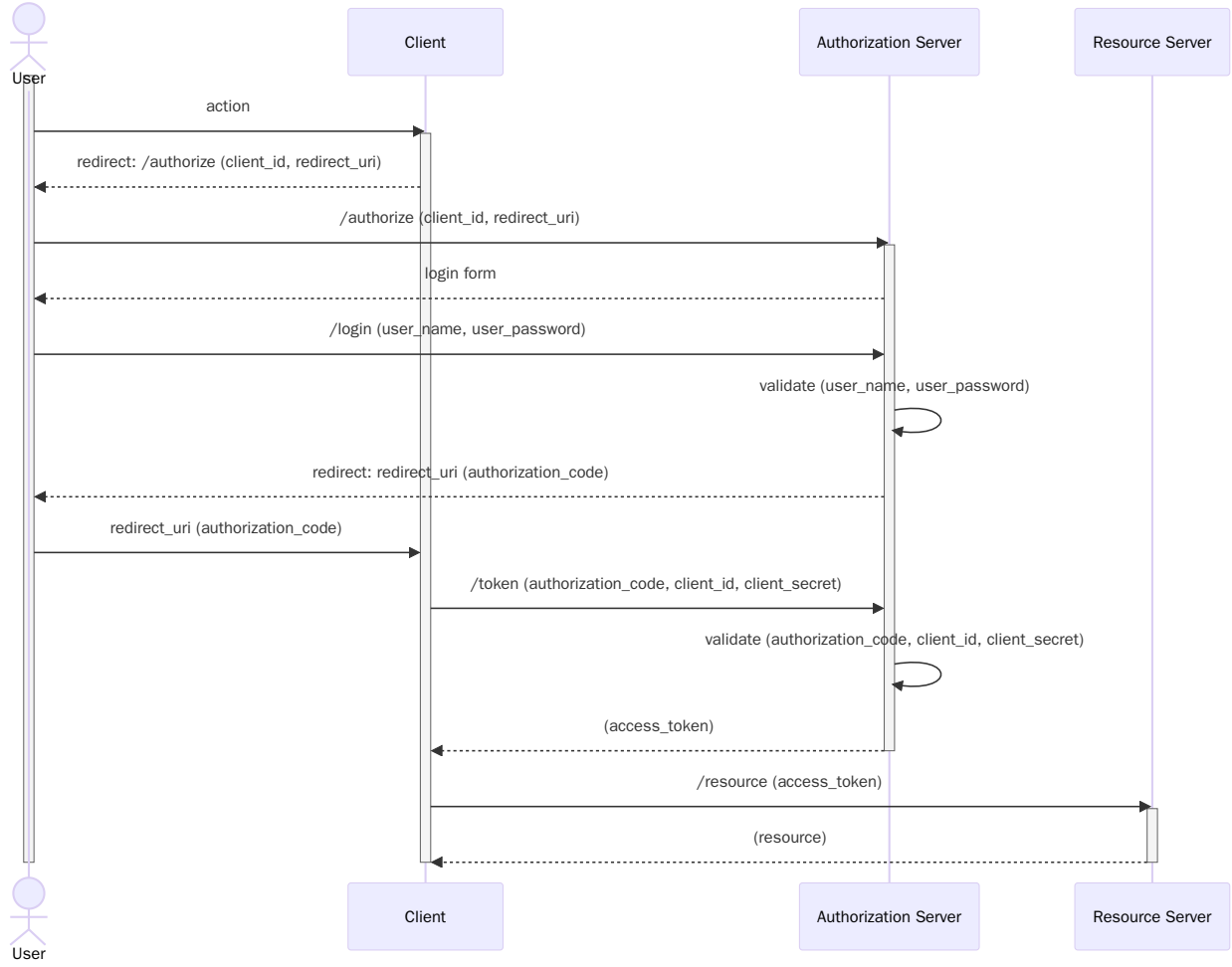
POLITECHNIKA GDAŃSKA

Resource Owner Password Credentials



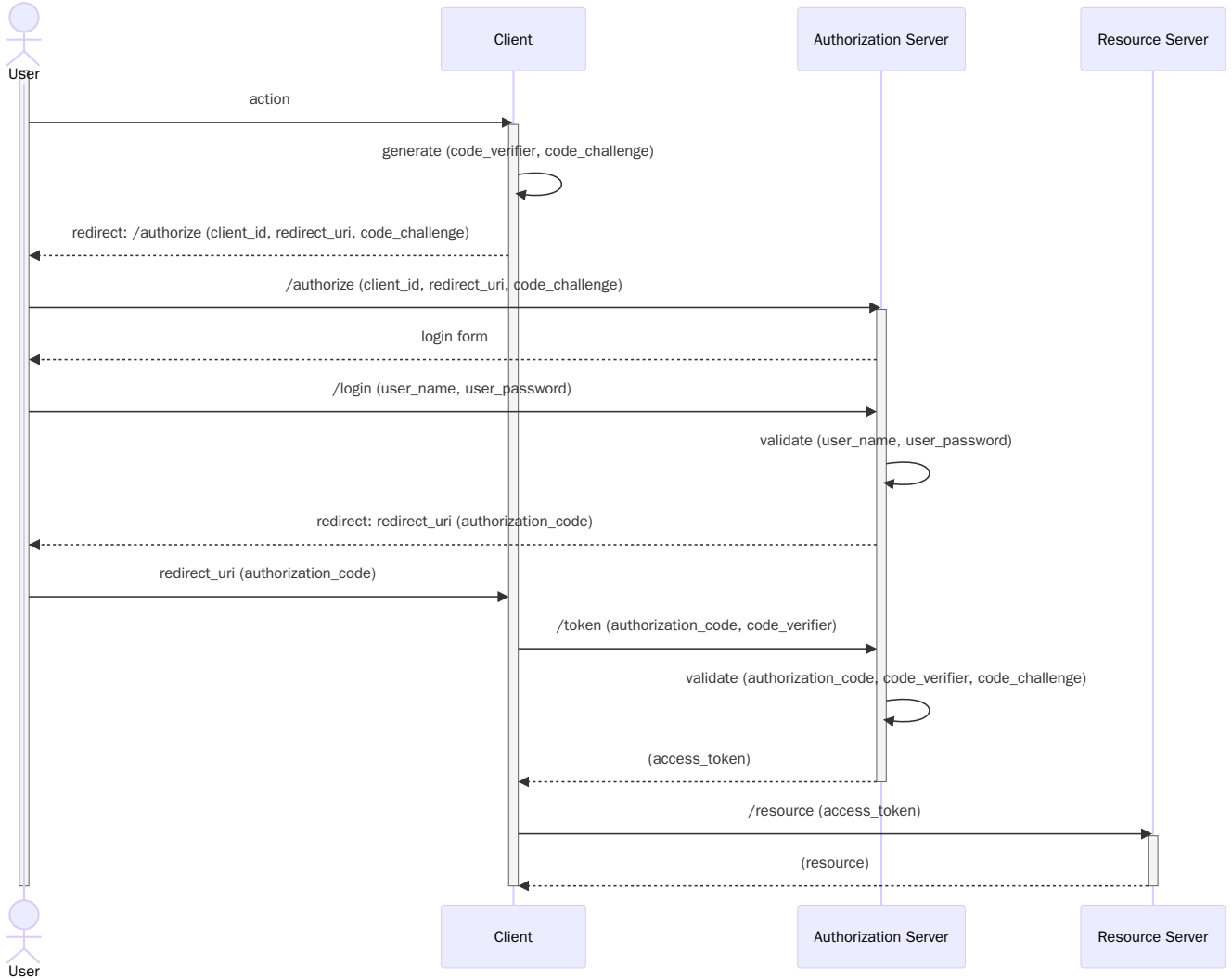
POLITECHNIKA GDAŃSKA

Authorization Code Flow



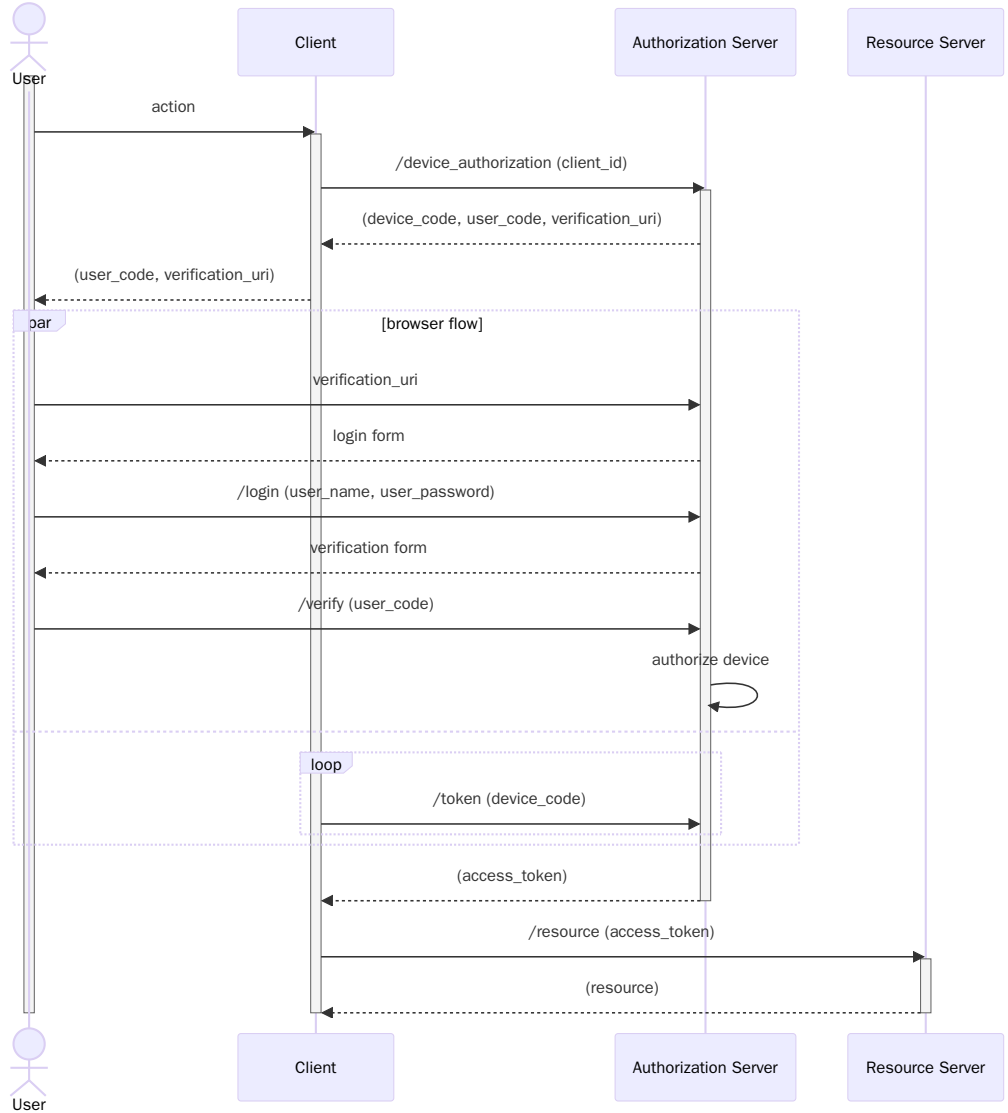
POLITECHNIKA GDAŃSKA

Authorization Code Flow with PKCE



POLITECHNIKA GDAŃSKA

Device Authorization Flow



Token JWT

JWT - *JSON Web Token*, otwarty standard bezpiecznego przekazywania informacji pomiędzy klientami a serwerem w postaci kompaktowego obiektu JSON. Token składający się z nagłówka, zawartości i podpisu.

Nagłówek:

```
{
  "typ" : "JWT",
  "alg" : "HS256"
}
```

Zawartość:

```
{
  "sub": "ookami",
  "exp": "1652297430"
}
```

Algorytmy podpisu:

- HS256 - HMAC SHA-256,
- RS256 - RSA SHA-256,
- ES256 - ECDSA SHA-256.

```
token = base64urlEncode(header) + "." + base64urlEncode(payload) + "." + sign(base64urlEncode(header) + "." + base64urlEncode(payload))
```

Token JWT - wpisy

Zarejestrowany wpisy (claims):

- `iss` - *Issuer*, wystawca tokena,
- `sub` - *Subject*, podmiot wystawionego tokenu,
- `aud` - *Audience*, odbiorca tokenu,
- `exp` - *Expiration*, data wygaśnięcia,
- `nbf` - *Not Before*, data, od kiedy token może być zaakceptowany,
- `iat` - *Issued At*, data wydania,
- `jti` - *JWT ID*, unikalny identyfikator.

Nagłówki:

- `alg` - *Algorithm*, algorytm podpisu (wymagany),
- `typ` - *Type*, typ tokenu
- `cty` - *Content Type*, format zawartości,
- `kid` - *Key ID*, identyfikator klucza użytego do podpisu.

Serwer autoryzacji

Niektóre znane serwery autoryzacji z obsługą OAuth2:'

- Keycloak,
- Auth0,
- Spring Authorization Server,
- Okta,
- Azure Active Directory,
- Google Identity Platform,
- ...

Bezpieczeństwo w Java EE

Kontener umożliwia dwa typy zapewniania bezpieczeństwa:

- **deklaratywne** – struktura zapewniająca bezpieczeństwo, w tym role, prawa dostępu i autoryzacja są zawarte poza logiką aplikacji przy pomocy adnotacji (i deskryptorów):
  - określa się przy pomocy wzorców, do jakich zasobów (stron) rola ma dostęp,
  - można zabezpieczyć dostęp do wybranych metod;
- **programistyczne** – zawarte w kodzie aplikacji:
  - programista ma dostęp do informacji na temat użytkownika (jego nazwa, grupa do której zależy).

Uwierzytelnianie w aplikacji webowej

Mechanizmy uwierzytelniania w aplikacjach bazujących na specyfikacji Servlet najczęściej wykorzystują dwa podejścia:

- basic auth - uwierzytelnianie na podstawie loginu oraz hasła przesłanych w nagłówku żądania,
- form auth - wysłania loginu oraz hasła poprzez formularz udostępniony przez aplikację,
- OpenID - uwierzytelnianie za pomocą dostawcy OpenID.

Basic HTTP authentication response header:

```
WWW-Authenticate: Basic realm="userRealm"
```

Basic HTTP authentication request header (admin/admin):

```
Authorization: Basic YWRtaW46YWRtaW4=
```

Mechanizmy uwierzytelniania

W ramach Security API dostępne adnotacje pozwalające na skonfigurowanie mechanizmów uwierzytelniania (bezpieczeństwo deklaratywne):

- `@BasicAuthenticationMechanismDefinition` - konfiguracja basic auth,
- `@FormAuthenticationMechanismDefinition` - konfiguracja formularza logowania obsługiwanego przez domyślny mechanizm,
- `@CustomFormAuthenticationMechanismDefinition` - konfiguracja formularza logowania obsługiwanego przez własny mechanizm,
- `@OpenIdAuthenticationMechanismDefinition` - konfiguracja uwierzytelniania za pomocą zewnętrznego dostawcy.

Uwierzytelnianie za pomocą formularza

```
@FormAuthenticationMechanismDefinition(
        loginToContinue = @LoginToContinue(
                loginPage = "/login.xhtml",
                errorPage = "/login_error.xhtml"
        )
)
@ApplicationScoped
public class Config {

}
```

Strony wskazane w adnotacji należy przygotować samodzielnie. Formularz logowania powinien odnosić się do wartości z dokumentacji:

```
<form action="j_security_check" method="post">
    <input name="j_username" type="text"/>
    <input name="j_password" type="password"/>
    <input type="submit"/>
</form>
```

Logowanie za pomocą formularza - kroki

Kroki związane z logowanie za pomocą formularza:

1. użytkownik próbuje uzyskać dostęp do zabezpieczonego zasobu,
2. serwer sprawdza, czy użytkownik jest zalogowany,
   - niezalogowany → krok 3,
   - zalogowany → krok 4;
3. wyświetlenie formularza logowania:
   - logowanie niepoprawne → strona błędu, KONIEC,
   - logowanie poprawne → krok 4;
4. serwer sprawdza role użytkownika:
   - uprawnienia nadane → krok 5,
   - brak uprawnień → 403, KONIEC;
5. serwer udostępnia zasób.

Logowanie za pomocą formularza - uwagi

Uwagi do logowania za pomocą formularza:

- najpierw występuje próba dostępu do zabezpieczonego zasobu,
- w reakcji na nią serwer wyświetla formularz logowania,
- użytkownik nie wchodzi bezpośrednio na stronę logowania:
  - użytkownik nie loguje się dla samego faktu bycia zalogowanym,
  - loguje się, gdy wymagają tego reguły dostępu do zasobu.

Własna obsługa logowania

Własna obsługa logowania:

- czasami przepływ sterowania dla logowania FORM nie odpowiada wymaganiom aplikacji, np.:
  - potrzebny formularz logowania na stronie głównej,
  - potrzebne logowanie użytkowników za pomocą mechanizmu AJAX,
  - konieczność wykonania dodatkowych operacji w czasie logowania:
  - np. weryfikacja czy użytkownik kliknął link aktywacyjny wysłany mailem;
- Servlet API umożliwia programistyczne logowanie użytkowników:
  - do wywołania z dowolnej własnej akcji.

Własna obsługa logowania - JSF

```
<h:form id="login-form">
    <h:panelGrid columns="2" styleClass="login-form">
        <h:outputText value="Login:"/>
        <h:inputText value="#{loginForm.login}"/>

        <h:outputText value="Password:"/>
        <h:inputSecret value="#{loginForm.password}"/>

        <h:commandButton action="#{loginForm.login()}" value="Login"/>
        <h:message for="login-form"/>
    </h:panelGrid>
</h:form>
```

Własna obsługa logowania - bean CDI

```
@Named
@ViewScoped
public class LoginForm {

    //...

    public void loginAction() throws IOException {
        Credential credential = new UsernamePasswordCredential(login, new Password(password));
        AuthenticationStatus status = securityContext.authenticate(
                request, response,
                AuthenticationParameters.withParams().credential(credential)
        );
        facesContext.responseComplete();
    }
}
```

Autoryzacja w servlecie

```
@ServletSecurity(
        value = @HttpConstraint(
                rolesAllowed = {"USER", "ADMIN"},
                transportGuarantee = ServletSecurity.TransportGuarantee.CONFIDENTIAL
        ),
        httpMethodConstraints = {
                @HttpMethodConstraint(rolesAllowed = {"ADMIN"}, value = "POST")
        }
)
public class UserServlet extends HttpServlet {

}
```

Zabezpieczenie servletu możliwe jest za pomocą adnotacji (bezpieczeństwo deklaratywne):

- `@ServletSecurity` - konfiguracja kontekstu bezpieczeństwa,
- `@HttpConstraint` - zabezpieczenie całego servletu:
  - `rolesAllowed` - role, w jakich musi być użytkownik,
  - `transportGuarantee` - wymóg SSL/TLS;
- `@HttpMethodConstraint` - zabezpieczenie pojedynczej metody HTTP:
  - `value` - nazwa metody HTTP,
  - `rolesAllowed` - role, w jakich musi być użytkownik.

Autoryzacja w pozostałych zasobach - role

Autoryzacja do pozostałych zasobów (np.: strony JSF) niereprezentowanych jako servlety musi być skonfigurowana w deskryptorze `web.xml` (bezpieczeństwo deklaratywne).

Deklaracja ról:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app>
    <security-role>
        <role-name>ADMIN</role-name>
    </security-role>

    <security-role>
        <role-name>USER</role-name>
    </security-role>
</web-app>
```

Autoryzacja w pozostałych zasobach

Autoryzacja dostępu:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app>
    <security-constraint>
        <web-resource-collection>
            <web-resource-name>user</web-resource-name>
            <url-pattern>/user/*</url-pattern>
        </web-resource-collection>

        <auth-constraint>
            <role-name>USER</role-name>
        </auth-constraint>

        <user-data-constraint>
            <transport-guarantee>CONFIDENTIAL</transport-guarantee>
        </user-data-constraint>
    </security-constraint>
</web-app>
```

Dostawy tożsamości w serwerze aplikacji

Dawniej domeny bezpieczeństwa były konfigurowane na poziomie serwera aplikacji. Aplikacja jedyne za pomocą odpowiednich interfejsów pobierała informacje o użytkowniku.

Połączenie aplikacji z domeną bezpieczeństwa wymaga konfiguracji w `web.xml`, plus ewentualnie w deskryptorze serwera.

```
<login-config>
    <auth-method>FORM</auth-method>
    <realm-name>userRealm</realm-name>
    <form-login-config>
        <form-login-page>/faces/login.xhtml</form-login-page>
        <form-error-page>/faces/error/login.xhtml</form-error-page>
    </form-login-config>
</login-config>
```

Dostawy tożsamości w aplikacji webowej

W aplikacji opartej na specyfikacji Jakarta Security można skorzystać z gotowych dostawców tożsamości:

@`@InMemoryIdentityStoreDefinition` - tożsamości użytkowników zapisane w pamięci,

- `@DatabaseIdentityStoreDefinition` - tożsamości użytkowników zapisane w bazie danych,
- `@LdapIdentityStoreDefinition` - tożsamości użytkowników zapisane w serwerze LDAP.

Konfiguracja dostawcy tożsamości

```
@DatabaseIdentityStoreDefinition(
        dataSourceLookup = "java:comp/env/jdbc/securityDS",
        callerQuery = "select password from users where user_name = ?",
        groupsQuery = "select group_name from groups where user_name = ?"
)
@ApplicationScoped
public class Config {

}
```

Konfigurując dostawcę tożsamości z bazy danych, należy podać:

- `dataSourceLookup` - adres JNDI do źródła danych skonfigurowanego na serwerze,
- `callerQuery` - zapytanie zwracające hasło na podstawie loginu, służy uwierzytelnianiu,
- `groupsQuery` - zapytanie zwracające grupy na podstawie loginu, służy autoryzacji.

user_name	password
admin	admin
user	user

user_name	group_name
admin	admin
admin	user
user	user

Przechowywanie haseł - konfiguracja

Należy pamiętać, że hasła użytkowników nie powinny być trzymane otwartym tekstem:

- standardowym rozwiązaniem jest przechowywanie skrótu (hash) hasła,
- podczas uwierzytelniania porównywane są skróty hasła dostarczonego przez użytkownika i tego przechowywanego w dostawcy.

Domyśla konfiguracja dla `@DatabaseIdentityStoreDefinition`:

```
@DatabaseIdentityStoreDefinition(
        dataSourceLookup = "java:comp/env/jdbc/securityDS",
        callerQuery = "select password from users where username = ?",
        groupsQuery = "select groupname from groups where username = ?",
        hashAlgorithm = Pbkdf2PasswordHash.class,
        hashAlgorithmParameters = {
                "Pbkdf2PasswordHash.Algorithm=PBKDF2WithHmacSHA256",
                "Pbkdf2PasswordHash.Iterations=2048",
                "Pbkdf2PasswordHash.SaltSizeBytes=32",
                "Pbkdf2PasswordHash.KeySizeBytes=32"
        }
)
@ApplicationScoped
public class Config {

}
```

Przechowywanie haseł - skrót

Skrót hasła należy wygenerować podczas zapisywania nowego (lub aktualizowania) użytkownika w dostawcy tożsamości.

```
@ApplicationScoped
public class UserService {

    @Inject
    private Pbkdf2PasswordHash pbkdf;

    public void save(User user) {
        pbkdf.initialize(Map.of(
                "Pbkdf2PasswordHash.Algorithm", "PBKDF2WithHmacSHA256",
                "Pbkdf2PasswordHash.Iterations", "2048",
                "Pbkdf2PasswordHash.SaltSizeBytes", "32",
                "Pbkdf2PasswordHash.KeySizeBytes", "32"
        ));
        user.setPassword(pbkdf.generate(user.getPassword().toCharArray()));
        // save user
    }

}
```

Autoryzacja dostępu do metod

Zabezpieczenie dostępu do stron czy servletów to nie wszystko. Należy również zabezpieczyć dostęp do logiki biznesowej:

- w beanach zarządzanych przez kontener CDI można skorzystać ze wstrzykniętego obiektu `jakarta.security.enterprise.SecurityContext`,
- `boolean SecurityContext#isCallerInRole(String)` pozwala sprawdzić, czy aktualny użytkownik posiada zdefiniowaną rolę (bezpieczeństwo programistyczne),
- `Principal SecurityContext#getCallerPrincipal()` pozwala pobrać standardowy obiekt reprezentujący użytkownika,
- `String Principal#getName()` pozwala na pobranie nazwy użytkownika.

We wcześniejszych wersjach wymagane były użycie obiektów przeznaczonych pod konkretne specyfikacje:

- `javax.ejb.EJBContext`,
- `javax.ws.rs.core.SecurityContext`,
- `javax.servlet.http.HttpServletRequest`,
- `javax.faces.context.ExternalContext`.

Autoryzacja w EJB (dawniej)

```java
import jakarta.annotation.Resource;
import jakarta.ejb.SessionContext;
import jakarta.ejb.Stateless;

@Stateless
public class MyEJB {

    @Resource
    private SessionContext sessionContext;

    public boolean isCallerInAdminRole() {
        return sessionContext.isCallerInRole("admin");
    }
}
```

Autoryzacja w JAX-RS (dawniej)

```java
import javax.ws.rs.core.SecurityContext;
import javax.ws.rs.core.Context;
import jakarta.ws.rs.Path;

@Path("/my-resource")
public class MyResource {

    @Context
    private SecurityContext securityContext;

    public boolean isCallerInAdminRole() {
        return securityContext.isUserInRole("admin");
    }

}
```

Autoryzacja w Servlet (dawniej)

```java
import jakarta.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/my-servlet")
public class MyServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
        boolean isCallerInAdminRole = request.isUserInRole("admin");
    }

}
```

Uwierzytelnianie w JSF (dawniej)

```java
import javax.faces.context.ExternalContext;
import javax.faces.context.FacesContext;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;

@ManagedBean
@RequestScoped
public class MyManagedBean {

    public boolean isCallerInAdminRole() {
        FacesContext facesContext = FacesContext.getCurrentInstance();
        ExternalContext externalContext = facesContext.getExternalContext;
        return externalContext.isUserInRole("admin");
    }

}
```

Autoryzacja dostępu do metod - przykład

```
@ApplicationScoped
public class UserService {

    @Inject
    private SecurityContext securityContext;

    public void save(User user) {
        if (securityContext.isCallerInRole(User.Roles.USER)) {
            //save user
        } else {
            throw new AccessControlException("Access denied");
        }
    }

}
```

Autoryzacja dostępu do metod w EJB

W beanach EJB dostępna jest adnotacja `@RolesAllowed` do zabezpieczania metod lub całych beanów.

```
import jakarta.annotation.security.RolesAllowed;
import jakarta.ejb.Stateless;

@Stateless
@DeclaredRoles({"admin", "user"})
public class SecuredBean {

    @RolesAllowed("admin")
    public String adminOnlyMethod() {
        return "Admin-only method executed.";
    }

}
```

Może być wymagane użycie `@DeclaredRoles` do zdefiniowania wszystkich dostępnych ról.

Autoryzacja dostępu do metod w JAX-RS

Adnotacja `@RolesAllowed` jest też przetwarzana przez beany JAX-RS.

```
import jakarta.annotation.security.RolesAllowed;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.core.Response;

@Path("/resource")
public class SecuredResource {

    @GET
    @Path("/admin")
    @RolesAllowed("admin")
    public Response adminOnlyMethod() {
        return Response.ok("Admin-only method executed").build();
    }

}
```

Zmiana poświadczeń EJB

```java
import jakarta.annotation.security.RunAs;
import jakarta.ejb.EJB;
import jakarta.ejb.Stateless;

@Stateless
@RunAs({"admin"})
public class RunAsBean {

    @EJB
    private SecuredBean bean;

    public String method() {
        return bean.adminOnlyMethod();
    }

}
```

Wykorzystanie adnotacji `@RunAs` pozwala na wymuszenie poświadczeń i ról, z jakimi zostanie wywołany kod. Wymaga konfiguracji w serwerze aplikacji oraz odwzorowania na istniejącego użytkownika.

Zmiana poświadczeń w Servlet

Konfigurację run-as można także zastosować w konfiguracji servletu, zarówno za pomocą adnotacji, jak i konfiguracji w deskryptorze,

```
<servlet>
    <servlet-name>run-as-servlet</servlet-name>
    <!-- ... -->
    <run-as>
        <role-name>admin</role-name>
    </run-as>
</servlet>
```

```
@WebServlet("/run-as-servlet")
@RunAs("admin")
public class RunAsServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

    }

}
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/security/3.0/jakarta-security-spec-3.0

Uzupełnienie informacji:

- https://eclipse-ee4j.github.io/jakartaee-tutorial/#security-2
- https://eclipse-ee4j.github.io/jakartaee-tutorial/#getting-started-securing-web-applications
- https://eclipse-ee4j.github.io/jakartaee-tutorial/#getting-started-securing-enterprise-applications
- https://eclipse-ee4j.github.io/jakartaee-tutorial/#using-jakarta-security
- https://eclipse-ee4j.github.io/jakartaee-tutorial/#jakarta-ee-security-advanced-topics
- https://www.baeldung.com/java-ee-8-security